

מסמך אבטחה בסיסית – HavuGym Crew

מכסה את סעיף 9 בבריף. כל טענה כאן מגובה בבדיקה. הפקודות לאימות מופיעות בסוף כל פרק.

1. עיקרון מנחה

האפליקציה אינה שכבת ההרשאות. בסיס הנתונים הוא.

זו לא הערה סגנונית אלא ההחלטה שממנה נגזר כל השאר. במבנה הרגיל, כל שאילתה חייבת לזכור `WHERE user_id =`?; שכחה אחת היא פרצה. כאן ההרשאות אכופות ב-Postgres, ולכן **שאיילתה שנשכח בה הסינון מחזירה אפס שורות, לא את כולן**. אותה טעות בדיוק – פרצה במקום אחד, באג ריק במקום השני.

שלוש שכבות בלתי תלויות:

1. GRANT – לאילו טבלאות ועמודות בכלל יש גישה
2. RLS – אילו שורות מתוך אלה
3. FUNCTION – מי רשאי לבצע פעולה מורכבת בכלל

פרק 8 מראה מקרה אמיתי שבו אחת מהן הייתה שגויה והשתיים האחרות החזיקו.

2. Authentication

Supabase Auth (GoTrue), אימייל וסיסמה.

היבט	מימוש
אחסון סיסמאות	bcrypt, אצל Supabase. המערכת שלנו לא רואה סיסמה בשום שלב
Session	JWT ב- <code>cookie, httpOnly, secure, sameSite=lax</code>
רענון	ב-middleware, בכל בקשה
אורך סיסמה	72-8 תווים
יצירת פרופיל	טריגר <code>on_auth_user_created</code> , בתוך טרנזקציית ההרשמה

2.1 למה 72 תווים כגבול עליון

bcrypt מתעלם מכל מה שמעבר ל-72 בתים. סיסמה בת 100 תווים הייתה **נחתכת בשקט**, והמשתמש היה מאמין שיש לו סיסמה חזקה יותר משיש לו. עדיף לסרב מאשר לחתוך בלי לומר.

2.2 מניעת account enumeration

התחברות כושלת מושלת מחזירה תמיד את אותה הודעה:

```
// מכוון עמוד: הבחנה בין "אין חשבון" ל-"סיסמה שגויה"
// למניית חשבונות-oracle הופכת את טופס ההתחברות ל
return actionError('Email or password is incorrect.');
```

2.3 יצירת פרופיל בטריגר, לא בקוד

```
create trigger on_auth_user_created
after insert on auth.users
for each row execute function public.handle_new_user();
```

הפרופיל והבנוס נוצרים בתוך אותה טרנזקציה כמו המשתמש. לא ייתכן משתמש בלי פרופיל, גם אם האפליקציה קרסה מיד אחרי ההרשמה.

Authorization .3

3.1 גבול ה-tenant

החבורה. כמעט כל שורה במערכת מגיעה רק דרך חברות בחבורה.

3.2 מלכודת הרקורסיה – והפתרון

ה-policy הטבעי על `havura_members` הוא "מותר לקרוא שורות של חבורה שאתה חבר בה" – שאילתה על `havura_members` מתוך policy על Postgres. `havura_members` מזהה את המעגל ומחזיר `42P17 infinite recursion`.

הפתרון:

```
create function public.is_havura_member(p_havura_id uuid)
returns boolean language sql
security definer
set search_path = public, pg_temp
stable as $$ ... $$;
```

בעצמו RLS-רץ כבעלים, ולכן לא כפוף ל' --
 'public' הקורא לא יכול להחליף את משמעות --

`set search_path` אינו קישוט. בלעדיו, קורא היה יכול ליצור schema משלו, לדחוף אותו ל-`search_path` ולגרום לפונקציה מיוחסת לפנות לטבלה שהוא שולט בה. כל פונקציית `SECURITY DEFINER` בפריקט מקבעת `search_path`.

3.3 מפת ההרשאות

תפקיד	הרשאות
anon	אפס. אין GRANT על אף טבלה
authenticated	SELECT לפי policy; UPDATE על עמודות נבחרות; EXECUTE על 5 פונקציות
service_role	הכול. קיים רק בשרת

3.4 מה מותר למשתמש מחובר

טבלה	קריאה	כתיבה
profiles	עצמי + חברי החבורות שלי	display_name , avatar_url בלבד
havras	חבורות שאני חבר בהן	עדכון/מחיקה – בעלים. INSERT – אין
havura_members	חברי החבורות שלי	מחיקה – שלי, או בעלים
workouts	של החבורות שלי	title , notes , performed_at , duration_min בלבד. INSERT – אין
workout_sets	דרך האימון	–
createine_ledger	שלי בלבד	אין
inventory , orders	שלי בלבד	דרך RPC / שרת
arrivals	של החבורות שלי	דרך RPC בלבד
exercises , shop_items	הכול	–

חברי החבורה רואים את האימונים שלך, את הדירוג שלך ומתי אתה במכון. הם לא רואים את הארנק שלך, את הרכישות שלך, ולא כמה שילמת לנו.

נוכחות היא מידע רגיש יותר מאימון שהסתיים – היא אומרת היכן אתה עכשיו. לכן היא מוגבלת לאותו גבול tenant בדיוק, פגה מעצמה, ומוצהרת ולא נמדדת ב-GPS: המערכת לעולם אינה מחזיקה את המיקום שלך, רק את העובדה שהצהרת שאתה בחדר כושר.

3.5 אימות

בדיקות, כל אחת כתובה כתקיפה 22 – tests/rls.integration.test.ts

4. פעולות שדורשות משתמש מחובר

כולן. הנתבים היחידים ללא session הם /api/health , /login , /api/stripe/webhook .

ה-middleware מפנה מבקר מנותק ל- /login . אבל זו נוחות, לא אבטחה – ההגנה האמיתית היא ש-RLS לא היה מחזיר לו כלום גם אילו הגיע לדף. אילו ה-middleware היה נמחק לגמרי, לא הייתה דליפה; היה מסך ריק.

הוכחה על הפריסה החיה:

```
$ curl -s -o /dev/null -w "%{http_code} -> %{redirect_url}" https://havugym-crew.vercel.app/feed
307 -> https://havugym-crew.vercel.app/login?next=%2Ffeed
```

5. מניעת גישה למידע של משתמש אחר

מעבר ל-RLS, ארבע החלטות ממוקדות:

5.1 אין חשיפה של `invite_code` ב-`SELECT`

אין `policy` שמאפשר לחפש חבורה לפי קוד. ההצטרפות עוברת דרך `join_havura()`, ולכן:

- אי אפשר למנות חבורות
- אי אפשר לאשש קוד מנוחש בלי להצטרף בפועל
- הקוד מתפקד כ-`capability`: קוד שגוי וקוד לא קיים אינם ניתנים להבחנה

5.2 404 ולא 403

דף אימון של חבורה זרה מחזיר "לא נמצא":

```
// לא נמצא "ו-"אין הרשאה" הן אותה תשובה בכוונה: 403 היה מאשר "  
// .הזה בחבורה של מישוהו אחר id-שקיים אימון עם ה  
if (!detail) notFound();
```

5.3 חשיפת פרופיל מותנית בחבורה משותפת

`shares_havura_with()` – אחרת האפליקציה הופכת לספריית משתמשים.

5.4 ה-cookie של החבורה הפעילה הוא רמז, לא סמכות

```
return memberships.find((m) => m.id === preferred) ?? memberships[0];
```

עריכת ה-cookie ב-devtools בוחרת רק מבין חבורות שאתה כבר חבר בהן.

6. ולידציית קלט

שלוש שכבות, ורק אחת מהן היא הגנה:

שכבה	היכן	תפקיד
HTML	דפדפן	משוב מידי. לא הגנה – נעקף בקלות
Zod	Server Action	הודעה קריאה לאדם
CHECK / RPC	Postgres	הקו האחרון

אילו Zod היה נעלם לגמרי, המסד עדיין היה דוחה חזרות שליליות, משקל מעל 500, אימון בלי סטים, קוד הזמנה בפורמט שגוי ויתרה שיורדת מתחת לאפס. Zod קיים כדי להפוך קלט שגוי להודעה – לא כדי להיות מה שמונע שורה מושחתת.

הגנה מפני SQL injection: אין קונקטנציה של SQL בשום מקום. הכול דרך PostgREST עם פרמטרים, ופונקציות מקבלות ארגומנטים מוקלדים. גם `p_sets` הוא `jsonb`, לא טקסט.

7. הגנה על קריאות API

7.1 אין API פנימי להגן עליו

זה העיצוב, לא מקרה. קריאות מתבצעות ב-Server Components; כתיבות ב-Server Actions. אין `/api/feed`, אין `/api/workouts`. שני ה-endpoints הציבוריים הם `/api/health` ו-`/api/stripe/webhook`, ושניהם קיימים כי גורם חיצוני חייב לקרוא להם.

7.2 Server Actions

נקודת הכניסה מוגנת ב-Origin check מובנה של Next.js נגד CSRF, וכל action מאמת מחדש את המשתמש בצד השרת. פרמטרים לעולם אינם נסמכים לזהות: `user_id` נלקח מ-`auth.uid()` בתוך המסד.

7.3 ה-webhook – ה-endpoint המסוכן ביותר

POST `/api/stripe/webhook` ציבורי לחלוטין: ל-Stripe אין session אצלנו. בלי אימות חתימה, כל אדם באינטרנט יכול היה לשלוח "התשלום הצליח" ולזכות את עצמו.

```
const payload = await request.text(); // ← RAW. חובה.  
event = getStripe().webhooks.constructEvent(  
  payload, signature, process.env.STRIPE_WEBHOOK_SECRET,  
);
```

שתי נקודות:

1. הגוף נקרא כטקסט גולמי. פענוח ל-JSON ואיחסון מחדש משנה את הבתים והחתימה נכשלת עבור כל אירוע לגיטימי – באג שנראה בדיוק כמו סוד שגוי.
2. הודעת השגיאה עמומה במכוון (`Invalid signature`). פירוט עוזר לתוקף לכייל ניסיונות זיוף ולא עוזר למשלב לגיטימי.

7.4 הסכום לעולם לא מגיע מהדפדפן

הלקוח שולח `slug` של חבילה. המחיר נקרא בשרת מ-`CREATINE_PACKS`, והזיכוי מתבצע רק אחרי ש-Stripe מאשרת `payment_status === 'paid'`. בקשה מזויפת יכולה לכל היותר לקנות חבילה אחרת במחיר האמיתי שלה.

8. אירוע אמיתי: הפרצה שנסגרה בגלל שכבה שנייה

זהו החלק החשוב במסמך, כי הוא מתאר טעות שקרתה בפועל.

מה קרה. מיגרציה `0002` הריצה:

```
revoke all on all tables in schema public from anon, authenticated;
```

ואז החזירה הרשאות בכוונה תחילה. אבל `ALL TABLES` חל רק על אובייקטים שקיימים באותו רגע, ושני ה-views נוצרו ב-`0006`, ארבע מיגרציות מאוחר יותר. ברירות המחדל של `(Supabase ALTER DEFAULT PRIVILEGES)` העניקו אותם אוטומטית ל-`anon` – וביטלו בשקט את העמדה שנקבעה ב-`0002`.

איך התגלה. `npm run db:verify`:

```
FAIL anon holds no table privileges -> weekly_user_stats, workout_feed
```

למה זו לא הייתה דליפה. ה-views הוגדרו עם `security_invoker = on`, ולכן הן רצות בהרשאות הקורא, ו-`anon` אין לו הרשאה על `workouts`:

```
GET /rest/v1/workout_feed (anon)
→ 401 42501 permission denied for table workouts
```

המסקנה. שתי בקורות בלתי תלויות; אחת הייתה שגויה; השנייה החזיקה. זו כל התועלת של `defence in depth`, וראוי לתאר אותה במפורש ולא לתקן בשקט.

התיקון (0008) – פעמיים:

```
revoke all on public.workout_feed from anon;
revoke all on public.weekly_user_stats from anon;
alter default privileges in schema public revoke all on tables from anon;
```

השורה השלישית היא התיקון האמיתי: היא מונעת מהאובייקט הבא שמישהו יוסיף לחזור על אותה טעות.

8.1 והערה על `security_invoker`

`view` רגיל רץ בהרשאות ה-`owner` – כלומר `postgres` – ולכן **עוקף RLS על כל טבלה שמתחתיו**. `view` שנכתב בברירת המחדל מעל `workouts` היה מגיש את האימונים של כל החבורות לכל מי שמורשה לקרוא מה-`view`. **זו עקיפת RLS מלאה שנוצרת מעצם הוספת `view`, בלי שאף `policy` השתנה.**

9. שני חורים שנסגרו בזמן הפיתוח

9.1 משתמש יכול היה לתת לעצמו ציון 100

ה-`policy` "עדכן את האימון שלך" נכון. ה-`GRANT` היה `grant update on workouts`, כלומר **על כל העמודות** – כולל `score`:

```
UPDATE workouts SET score = 100 WHERE id = <id>; -- היה עובר
```

התיקון אינו `policy` טובה יותר אלא הרשאה ברמת **עמודה**:

```
grant update (title, notes, performed_at, duration_min) on workouts to authenticated;
```

ל-`score` פשוט אין הרשאת `UPDATE`. **מותר לתקן מה שרשמת. אסור לתת לעצמך ציון.**

9.2 אותו דבר על היתרה

UPDATE grant update (display_name, avatar_url) on profiles — create_balance
profiles SET create_balance = 999999 policy בשום תלות בשום.

שתי הבדיקות:

```
tests/rls.integration.test.ts > a member cannot award themselves a score  
tests/rls.integration.test.ts > a member cannot mint their own createine
```

10. שמירת סודות

סוד	היכן	חשוף לדפדפן?
NEXT_PUBLIC_SUPABASE_URL	env	כן – ובכונה
NEXT_PUBLIC_SUPABASE_ANON_KEY	env	כן – ובכונה
SUPABASE_SERVICE_ROLE_KEY	env	לא
STRIPE_SECRET_KEY	env	לא
STRIPE_WEBHOOK_SECRET	env	לא
סיסמאות משתמשים	אצל Supabase, bcrypt	לא נראות לנו כלל
פרטי כרטיס אשראי	אצל Stripe בלבד	לא עוברים דרכנו

10.1 למה מפתח ה-anon בטוח בדפדפן

הוא אינו סוד, והוא חסר סמכות משלו. כל בקשה שנשלחת איתו עדיין עוברת דרך RLS. הוא מזהה את הפרויקט; הוא אינו מעניק גישה. מפתח ה-service role לעומת זאת עוקף RLS לחלוטין – לכן הוא מעולם לא נושא תחילת NEXT_PUBLIC_.

10.2 הגנה מפני הטעות הגרועה ביותר

ייבוא של admin.ts ל-Client Component היה שולח את מפתח ה-service role לכל דפדפן. שתי הגנות:

```
import 'server-only'; // הופך ייבוא כזה לשגיאת build  
  
if (typeof window !== 'undefined') {  
  throw new Error('createAdminClient() was called in the browser...');  
}
```

10.3 היגיינה תפעולית

- .env.local ו-.env.vercel ב-gitignore. מהקומיט הראשון
- .env.example מתועד עם ערכים ריקים
- אף סוד לא נכנס להיסטוריית git – ניתן לאימות: git log -p | grep sb_secret

מגבלה שראוי להצהיר עליה: במהלך הפיתוח מפתח ה-service role הודבק בשיחת עבודה. הוא ניתן להחלפה בלחיצה אחת בלוח הבקרה של Supabase, וזה מה שצריך לעשות לפני שהמאגר נפתח לציבור. הכלי מוגן; ההרגל הוא זה שנכשל.

11. סיכונים שנותרו

#	סיכון	חומרה	הפחתה קיימת	מה הייתי עושה
1	אין rate limiting משלנו	בינונית	הגבלות Supabase; אין endpoint לקריאה	Vercel firewall; token bucket Postgres-ב
2	קוד הזמנה בן 6 תווים (32 ⁶ ≈ 1.07 מיליארד)	נמוכה	אין SELECT לאישוש; רק ניחוש עיוור	הגבלת קצב על join_havura; פקיעה
3	סגירת שבוע רצה בטעינת דף	נמוכה	idempotent; תשלום כפול בלתי אפשרי	pg_cron
4	אין 2FA	נמוכה	—	Supabase MFA
5	אין audit log לפעולות מנהל	נמוכה	ה-ledger מתעד את המטבע במלואו	טבלת audit
6	ה-service role חי ב-env של Vercel	מובנה ל-stack	לא ב-bundle, לא ב-git	רוטציה תקופתית
7	אימות אימייל מושבת – כתובת לא מאומתת	בינונית	סיסמה נדרשת; אין השפעה על בידוד בין חברות	SMTP חיצוני, ואז החזרת האימות
8	לא מומש soft delete	נמוכה	מחיקה מוגבלת לשלי	deleted_at
11	אין מסך "מחק את החשבון שלי"	בינונית	המחיקה אפשרית ברמת המסד (ראו 11.1); חסר רק מסך	פעולת מחיקה עצמית
9	אין CSP מפורש	נמוכה	אין dangerouslySetInnerHTML; React בורח מטקסט	CSP כותרות ב-next.config
10	תוכן משתמש (כותרת, הערות) מוצג	נמוכה	React בורח כברירת מחדל	—

11.1 פגם שנמצא ותוקן: מחיקת חשבון הייתה חסומה

בזמן QA התגלה שמחיקת משתמש נכשלת עם Database error deleting user ו-500 – עבור **כל מי שאי פעם יצר חבורה**. הסיבה: כל מפתח זר שמצביע על profiles הוגדר ON DELETE CASCADE חוץ מ-havuras.created_by, שנשאר בברירת המחדל RESTRICT.

זהו פגם פרטיות ולא רק פגם נתונים: משתמש שמבקש שיימחק לא היה יכול להימחק.

התיקון הנאיבי היה מסוכן בהרבה. CASCADE היה גורם לכך שמחיקת המייסד תמחק את החבורה, ושרשור החבורה היה מוחק את האימונים של כל שאר החברים – כלומר, אדם אחד שמממש את זכותו למחיקה היה מוחק את הנתונים של תשעה אנשים אחרים. SET NULL (מיגרציה 0009) הוא המודל הנכון: החבורה שורדת את החשבון שפתח אותה, והבעלות האמיתית ממילא נשמרת ב-havura_members.

12. אימות עצמאי

```
# 12 – אינצוריאנטים ל RLS – פועיל, ל anon-הרשאות  
# ledger-שווה ל cache-ועל היתרה, ה score על UPDATE אין  
npm run db:verify  
  
# 22 תקיפות מול המסד האמיתי  
npm run test:integration  
  
# הפריסה הזויה מצהירה שהיא עדיין מסרבת לאנונימי  
curl -s https://havugym-crew.vercel.app/api/health | jq .rls  
# { "anonReadStatus": 401, "anonIsDenied": true }
```

נכשל כאשר האבטחה **מתרופפת** – הכיוון שחשוב. אם מישוהו יעניק ל-`anon` הרשאת קריאה בפרודקשן, גם החבילת E2E נופלת.